

Politecnico di Milano  
Embedded Systems Final Report

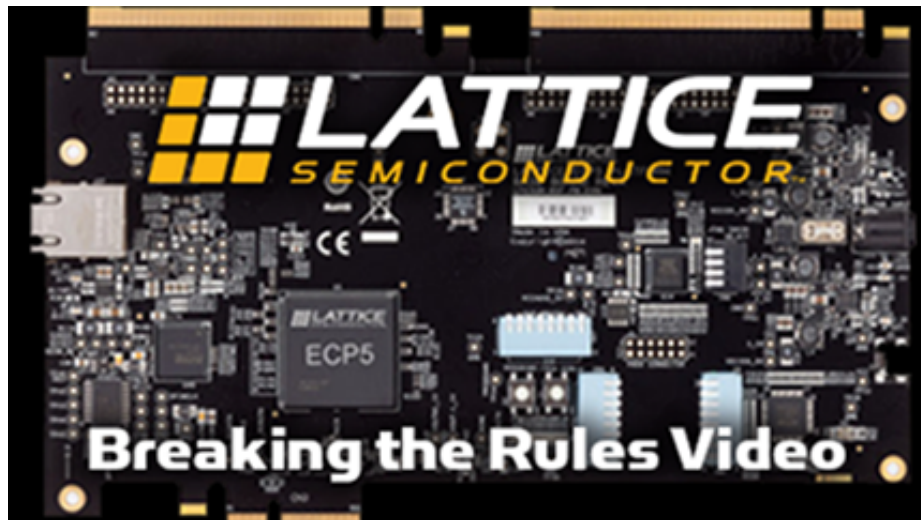
# **Real-Time Image Processing and Tiny CNN Integration on a Lattice ECP5 FPGA**

**Students:** Ali Edareh Heidarabadi and Mostafa  
Moazenkakhki

**Student IDs:** 11036028 and 11033591

**Supervisor :** Prof. Serena Curzel

**Academic Year:** 2025–2026



Project target platform and real-time video-processing pipeline.

# Abstract

This project investigates the integration of real-time image processing and lightweight neural-network inference on an embedded FPGA platform. The target platform is the Lattice ECP5 EVDK board based on the ECP5UM-85F FPGA device. The starting point is an existing dual CSI-2 camera to HDMI bridge reference design. The project modifies this video pipeline by inserting custom processing blocks and by evaluating the feasibility of adding a small convolutional neural network (CNN) for person/non-person classification.

The first part of the work focuses on incremental video-pipeline modifications. Simple hardware blocks such as grayscale conversion, thresholding, Sobel edge detection, and bypass selection are introduced to validate the integration methodology. These blocks are important because they provide observable visual outputs through HDMI and allow debugging of synchronization, pixel timing, and data-flow behavior before introducing a more complex accelerator.

The second part of the project explores the deployment of a tiny CNN on the FPGA. A compact model is trained in Python/Keras using grayscale images and later converted to fixed-point arithmetic. The network is intentionally small in order to reduce the consumption of LUTs, registers, embedded block RAMs, and DSP multipliers. The fixed-point model is compared against the floating-point Keras baseline to verify that the numerical degradation remains acceptable.

The hardware implementation flow uses hls4ml and Bambu high-level synthesis to generate RTL for the neural-network inference module. The generated logic is then integrated into the Lattice Diamond project together with the existing video-processing design. Although the design is able to pass several stages of the tool flow, the complete system encounters routing congestion and negative timing slack during place-and-route. This result is significant because it shows that fitting a neural network in terms of resource percentage is not sufficient; routing complexity, timing closure, clock-domain behavior, and pipeline architecture are equally important constraints in FPGA-based embedded vision systems.

The final outcome of the project is therefore both an implementation attempt and a fea-

sibility analysis. The project demonstrates that simple stream-based image-processing functions can be added to the ECP5 video pipeline, while CNN integration requires further architectural optimization. The main lessons concern incremental hardware integration, fixed-point design, high-level synthesis limitations, and the difference between algorithmic feasibility and physical implementation feasibility on a constrained FPGA.

**Keywords:** FPGA, ECP5, embedded vision, HDMI, CSI-2, image processing, CNN, hls4ml, Bambu HLS, fixed-point arithmetic, timing closure.

# Contents

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Abstract</b>                                      | <b>i</b>  |
| <b>1 Introduction</b>                                | <b>1</b>  |
| 1.1 Context . . . . .                                | 1         |
| 1.2 Project Goal . . . . .                           | 2         |
| 1.3 Main Contributions . . . . .                     | 2         |
| 1.4 Report Organization . . . . .                    | 3         |
| <b>2 Technical Background</b>                        | <b>4</b>  |
| 2.1 FPGA-Based Video Processing . . . . .            | 4         |
| 2.2 Convolutional Neural Networks on FPGAs . . . . . | 4         |
| 2.3 Fixed-Point Arithmetic . . . . .                 | 5         |
| 2.4 High-Level Synthesis . . . . .                   | 5         |
| 2.5 Timing Closure and Routing Congestion . . . . .  | 6         |
| <b>3 Hardware and Software Platform</b>              | <b>7</b>  |
| 3.1 Target FPGA Board . . . . .                      | 7         |
| 3.2 ECP5 Resource Considerations . . . . .           | 8         |
| 3.3 Software Toolchain . . . . .                     | 8         |
| 3.4 Design Methodology . . . . .                     | 9         |
| <b>4 Baseline Camera-to-HDMI Pipeline</b>            | <b>10</b> |
| 4.1 Reference Design Overview . . . . .              | 10        |
| 4.2 Pipeline Insertion Point . . . . .               | 11        |
| 4.3 Signal Preservation . . . . .                    | 12        |
| 4.4 Debugging Strategy . . . . .                     | 12        |

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>5</b> | <b>Implemented Image-Processing Blocks</b>            | <b>13</b> |
| 5.1      | Motivation for Simple Processing Blocks . . . . .     | 13        |
| 5.2      | Bypass Mode . . . . .                                 | 13        |
| 5.3      | Grayscale Conversion . . . . .                        | 14        |
| 5.4      | Thresholding . . . . .                                | 15        |
| 5.5      | Sobel Edge Detection . . . . .                        | 15        |
| 5.6      | Lessons from Image-Processing Integration . . . . .   | 16        |
| <b>6</b> | <b>Tiny CNN for Person Detection</b>                  | <b>17</b> |
| 6.1      | Problem Formulation . . . . .                         | 17        |
| 6.2      | Dataset Preparation . . . . .                         | 17        |
| 6.3      | Model Architecture . . . . .                          | 18        |
| 6.4      | Keras Model Definition . . . . .                      | 18        |
| 6.5      | Training Results . . . . .                            | 19        |
| 6.6      | Why the Model Must Be Small . . . . .                 | 20        |
| <b>7</b> | <b>Fixed-Point Conversion and Hardware Generation</b> | <b>21</b> |
| 7.1      | Need for Quantization . . . . .                       | 21        |
| 7.2      | Precision Selection . . . . .                         | 21        |
| 7.3      | Floating-Point vs Fixed-Point Agreement . . . . .     | 22        |
| 7.4      | hls4ml Conversion Flow . . . . .                      | 22        |
| 7.5      | Bambu HLS Flow . . . . .                              | 25        |
| 7.6      | Generated Interface . . . . .                         | 25        |
| <b>8</b> | <b>FPGA Integration</b>                               | <b>27</b> |
| 8.1      | Overall Integrated Architecture . . . . .             | 27        |
| 8.2      | Patch Extraction . . . . .                            | 28        |
| 8.3      | Latency Alignment . . . . .                           | 28        |
| 8.4      | Clocking and Reset . . . . .                          | 29        |
| 8.5      | Generated RTL Integration . . . . .                   | 30        |
| 8.6      | Overlay Output . . . . .                              | 31        |

# List of Figures

|     |                                                                                                                                                                                                                                                                                   |    |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | Original reference pipeline before custom modifications. . . . .                                                                                                                                                                                                                  | 10 |
| 4.2 | Modified pipeline with custom processing insertion point. . . . .                                                                                                                                                                                                                 | 11 |
| 7.1 | Conceptual interface between the streaming video pipeline and CNN inference. The post-ISP RGB stream is converted into a downsampled grayscale patch, buffered in BRAM, streamed to the CNN accelerator, and converted into a detection result used by the overlay logic. . . . . | 26 |
| 8.1 | Target final architecture of the embedded vision system, including the original video-processing pipeline, custom preprocessing blocks, the HLS-generated CNN accelerator, and the HDMI output stage. . . . .                                                                     | 27 |

# List of Tables

|     |                                                                                                      |    |
|-----|------------------------------------------------------------------------------------------------------|----|
| 3.1 | Target platform summary. . . . .                                                                     | 7  |
| 3.2 | Observed resource usage in the final implementation attempt. . . . .                                 | 8  |
| 3.3 | Software tools used in the project. . . . .                                                          | 9  |
| 6.1 | Dataset summary. . . . .                                                                             | 17 |
| 6.2 | Tiny CNN architecture used for FPGA-oriented person classification. .                                | 18 |
| 6.3 | Confusion matrix obtained on the evaluation set using a classification<br>threshold of 0.50. . . . . | 19 |
| 6.4 | Representative classification metrics. . . . .                                                       | 19 |
| 7.1 | Representative fixed-point precision configuration. . . . .                                          | 22 |
| 7.2 | Representative floating-point vs fixed-point comparison. . . . .                                     | 22 |



# Chapter 1

## Introduction

### 1.1 Context

Embedded computer vision is increasingly used in applications such as surveillance, human detection, driver assistance, industrial inspection, smart cameras, and edge artificial intelligence. In many of these applications, sending all raw visual data to a remote server is not ideal. It can increase latency, consume bandwidth, expose privacy-sensitive information, and require a continuous network connection. For this reason, there is strong motivation to move at least part of the processing close to the sensor.

Field-programmable gate arrays (FPGAs) are attractive for this type of workload because they can implement custom data paths that operate directly on streaming pixels. Compared with CPUs, FPGAs can exploit fine-grained spatial parallelism. Compared with GPUs, they can achieve deterministic latency and lower power consumption for carefully designed pipelines. However, FPGA design also introduces constraints that are less visible in software development. These constraints include limited logic resources, fixed memory structures, finite DSP blocks, routing congestion, strict timing closure, and the need to manage pixel synchronization at the hardware level.

The project described in this report is based on a Lattice ECP5 EVDK platform and an existing CSI-2 camera to HDMI bridge design. The reference design already implements a working video path from camera input to display output. Instead of creating an entire video system from zero, the project uses the existing reference pipeline and modifies it by adding processing blocks. This is a realistic embedded-systems scenario: a developer often starts from a vendor or laboratory reference design and then customizes it for a specific application.

## 1.2 Project Goal

The main objective of the project is to explore whether a lightweight visual-processing and person-detection system can be integrated into an FPGA-based real-time video pipeline. The goal can be divided into four smaller objectives:

- O1.** Understand the structure of the existing CSI-2 camera to HDMI reference design.
- O2.** Insert simple stream-based image-processing modules into the video pipeline.
- O3.** Train and convert a tiny CNN suitable for FPGA inference.
- O4.** Evaluate the feasibility of integrating the CNN accelerator into the complete ECP5 implementation flow.

The project is not only about achieving a final demonstration. It is also about understanding the engineering barriers that appear when a machine-learning model is moved from a software environment to an FPGA video pipeline. In software, the model is evaluated using accuracy, precision, recall, and loss. In hardware, the same model must also satisfy fixed-point precision constraints, throughput requirements, memory constraints, synthesis constraints, routing constraints, and clock-frequency constraints.

## 1.3 Main Contributions

The main contributions of this work are the following:

- An analysis of the original ECP5 camera-to-HDMI video pipeline.
- A step-by-step integration methodology for adding custom image-processing blocks.
- A small CNN architecture designed for binary person/non-person classification.
- A fixed-point conversion and software/hardware consistency analysis.
- A synthesis and implementation evaluation of the generated hardware module.
- A discussion of routing congestion and timing failure as practical FPGA design limitations.

The final result should be interpreted as a feasibility-oriented embedded-systems project. The simple video-processing blocks represent successful incremental integration, while the CNN part exposes the limitations of the chosen architecture and tool flow when targeting a constrained FPGA device.

## 1.4 Report Organization

This report is organized as follows. Chapter 2 introduces the technical background of FPGA-based image processing, CNN inference, fixed-point arithmetic, and high-level synthesis. Chapter 3 describes the hardware and software platform. Chapter 4 explains the original reference design and the position where custom modules are inserted. Chapter 5 describes the implemented image-processing blocks. Chapter 6 presents the CNN model and its training. Chapter 7 discusses fixed-point conversion and hardware generation. Chapter 8 explains the integration into the FPGA project. Chapter 8.6 presents implementation results. Chapter ?? discusses problems and design trade-offs. Chapter ?? concludes the report and proposes future improvements.

# Chapter 2

## Technical Background

### 2.1 FPGA-Based Video Processing

A video-processing pipeline implemented on an FPGA is usually organized as a stream of pixels. Each clock cycle, one or more pixels enter the processing chain, pass through a sequence of hardware blocks, and finally leave the device toward a display interface or memory buffer. This model is different from software image processing, where an image is typically stored as a complete array in memory and processed by nested loops.

The stream-based model has important advantages. It avoids storing the entire frame when not necessary, reduces memory bandwidth, and allows the pipeline to produce one output pixel per clock cycle after the initial latency. However, it also makes the design sensitive to timing and synchronization. A hardware block must preserve not only pixel values, but also control signals such as frame valid, line valid, horizontal synchronization, vertical synchronization, data enable, and sometimes pixel coordinates.

In this project, the initial processing blocks are intentionally simple. Grayscale conversion, thresholding, and Sobel edge detection are suitable first targets because they operate directly on pixels or small neighborhoods. They can be inserted into a video stream and observed visually through HDMI. This makes them useful for debugging before integrating a more complex machine-learning accelerator.

### 2.2 Convolutional Neural Networks on FPGAs

Convolutional neural networks are widely used for image classification and object detection. A CNN applies convolution filters to local image regions and learns hierarchical feature representations. On conventional processors, CNN inference is expressed as a sequence of tensor operations. On an FPGA, these operations must be mapped to hardware structures such as multipliers, adders, line buffers, registers, memories, and

control finite-state machines.

The difficulty of implementing CNNs on small or mid-range FPGAs comes from several factors:

- Convolution requires many multiply-accumulate operations.
- Feature maps and intermediate activations require memory.
- Parallel implementations consume more resources but reduce latency.
- Serial implementations save area but increase latency and control complexity.
- Fixed-point arithmetic must be used carefully to avoid unacceptable accuracy loss.
- Generated RTL can be difficult to route if the data path is large or poorly pipelined.

For these reasons, the CNN used in this project is very small. The purpose is not to compete with modern object detectors, but to study the process of moving a compact neural model into an FPGA design.

## 2.3 Fixed-Point Arithmetic

Software neural networks are usually trained using floating-point arithmetic. Floating-point operations are flexible but expensive in FPGA logic, especially when many operations must happen in parallel. Fixed-point arithmetic represents real numbers using a fixed number of integer and fractional bits. For example, a fixed-point type with 16 total bits and 5 integer bits can represent a limited range with a fixed resolution.

The general fixed-point design problem is to choose enough bits to preserve model behavior while avoiding unnecessary hardware cost. If the bit width is too small, values may saturate, overflow, or lose precision. If the bit width is too large, the design consumes more LUTs, registers, memory, and routing resources. In this project, several fixed-point precisions were tested during the model-conversion stage. The selected precision is a compromise between numerical agreement and hardware cost.

## 2.4 High-Level Synthesis

High-level synthesis (HLS) converts a behavioral description, often written in C or C++, into register-transfer level (RTL) hardware. HLS can accelerate design exploration

because the developer can describe an algorithm at a higher level than Verilog or VHDL. However, HLS does not remove the need to understand hardware. The generated RTL still has area, timing, memory, and interface constraints.

The HLS flow used in this project is based on hls4ml and Bambu. hls4ml is used to translate machine-learning models into synthesizable HLS-oriented code. Bambu is then used to generate hardware from the C/C++ description. The generated output can be integrated into a larger FPGA project. This flow is useful for rapid prototyping, but the final design quality depends strongly on model architecture, precision, interfaces, pipelining, and tool settings.

## 2.5 Timing Closure and Routing Congestion

A common misunderstanding in FPGA projects is to assume that a design will work if the resource utilization percentages are below 100%. In practice, this is not enough. The place-and-route tool must physically place all logic elements on the FPGA and connect them using routing resources. If the design has many long interconnections, dense clusters, or insufficient pipelining, it may fail to route or may route with negative timing slack.

Timing slack measures the margin between the required arrival time and the actual arrival time of a signal. Negative slack means that some paths are too slow for the target clock period. Routing congestion means that the available routing channels are not sufficient or not convenient for the required interconnections. In the final stage of this project, the design showed acceptable resource utilization but still encountered routing and timing problems. This is an important outcome and is discussed in detail in later chapters.

# Chapter 3

## Hardware and Software Platform

### 3.1 Target FPGA Board

The target hardware platform is the Lattice ECP5 EVDK board, which includes a Lattice ECP5UM-85F FPGA. The ECP5 family is designed for low-power and cost-sensitive applications while still providing logic resources, embedded memory, DSP blocks, and high-speed interfaces. In this project, the FPGA is used as a real-time video-processing device. The board receives video from camera interfaces and outputs processed video through HDMI.

“‘latex

Table 3.1: Target platform summary.

| Component         | Description                                          |
|-------------------|------------------------------------------------------|
| Development kit   | Lattice Embedded Vision Development Kit (EVDK)       |
| Processing FPGA   | Lattice ECP5 FPGA                                    |
| Target device     | LFE5UM-85F-BG756                                     |
| Input interface   | Dual MIPI CSI-2 camera input                         |
| Output interface  | HDMI output                                          |
| Reference design  | Dual CSI-2 Camera to HDMI Bridge                     |
| Development tools | Lattice Diamond + Synplify Pro                       |
| ML / HLS flow     | Python, Keras, hls4ml, Bambu HLS                     |
| Application       | Real-time image processing and FPGA CNN acceleration |

## 3.2 ECP5 Resource Considerations

The ECP5UM-85F device provides a finite number of LUTs, registers, embedded block RAMs, and DSP multipliers. These resources must be shared between the existing camera-to-HDMI design and the added processing modules. A CNN accelerator can consume all four categories: LUTs for control and arithmetic, registers for pipeline stages, embedded memories for buffers and weights, and DSPs for multiplications.

In the final implementation attempt, the reported resource usage was approximately as shown in Table 3.2. The numbers indicate that the design did not exceed the nominal resource limits. However, the place-and-route stage at first attempt failed because routing and timing constraints were not satisfied.

Table 3.2: Observed resource usage in the final implementation attempt.

| Resource  | Used   | Approx. Utilization | Comment                       |
|-----------|--------|---------------------|-------------------------------|
| LUT4      | 24,501 | 29%                 | Moderate logic usage          |
| Registers | 12,541 | 14%                 | Moderate sequential usage     |
| EBR       | 33     | 16%                 | Used for buffers / memories   |
| DSP       | 12     | 8%                  | Used by arithmetic operations |

This table is important because it shows the difference between area feasibility and physical implementation feasibility. Even though the area numbers appear acceptable, the design can still be too difficult to route if the interconnection structure is complex.

## 3.3 Software Toolchain

The project uses a mixed software and hardware toolchain. Neural-network training is performed in Python, while FPGA implementation is performed using vendor and HLS tools.



Table 3.3: Software tools used in the project.

| Tool               | Role                                | Notes                               |
|--------------------|-------------------------------------|-------------------------------------|
| Python             | Data preparation and model training | Used with NumPy/Keras               |
| Keras / TensorFlow | CNN training                        | Floating-point baseline model       |
| hls4ml             | Model-to-HLS conversion             | Fixed-point conversion and HLS code |
| Bambu HLS          | C/C++ to RTL generation             | Generates hardware implementation   |
| Synplify Pro       | Logic synthesis                     | Used through the Lattice flow       |
| Lattice Diamond    | FPGA implementation                 | Map, place, route, timing analysis  |

### 3.4 Design Methodology

The methodology followed in the project is incremental. Instead of integrating the CNN immediately, the pipeline is modified in stages. This reduces risk and makes debugging easier.

1. Start from the working camera-to-HDMI design.
2. Identify the pixel-stream location after basic color conversion / gamma stages.
3. Insert a bypass path to verify that the modified design still works.
4. Add grayscale conversion and check visual output.
5. Add thresholding and Sobel filtering as intermediate processing blocks.
6. Train and test a tiny CNN in software.
7. Convert the CNN to fixed point and compare with the original model.
8. Generate RTL using the HLS flow.
9. Integrate the generated module into the FPGA project.
10. Run synthesis, map, place-and-route, and timing analysis.

This methodology is valuable because each stage produces an observable result. If a later stage fails, the previous working stage can be used as a reference.

# Chapter 4

## Baseline Camera-to-HDMI Pipeline

### 4.1 Reference Design Overview

The starting point of this project is a dual CSI-2 camera to HDMI bridge design. The reference design receives image data from camera sensors, performs the required format conversion and video timing operations, and drives an HDMI output. The original design already contains several important components, including camera input handling, pixel formatting, color-processing stages, video timing generation, and output serialization.

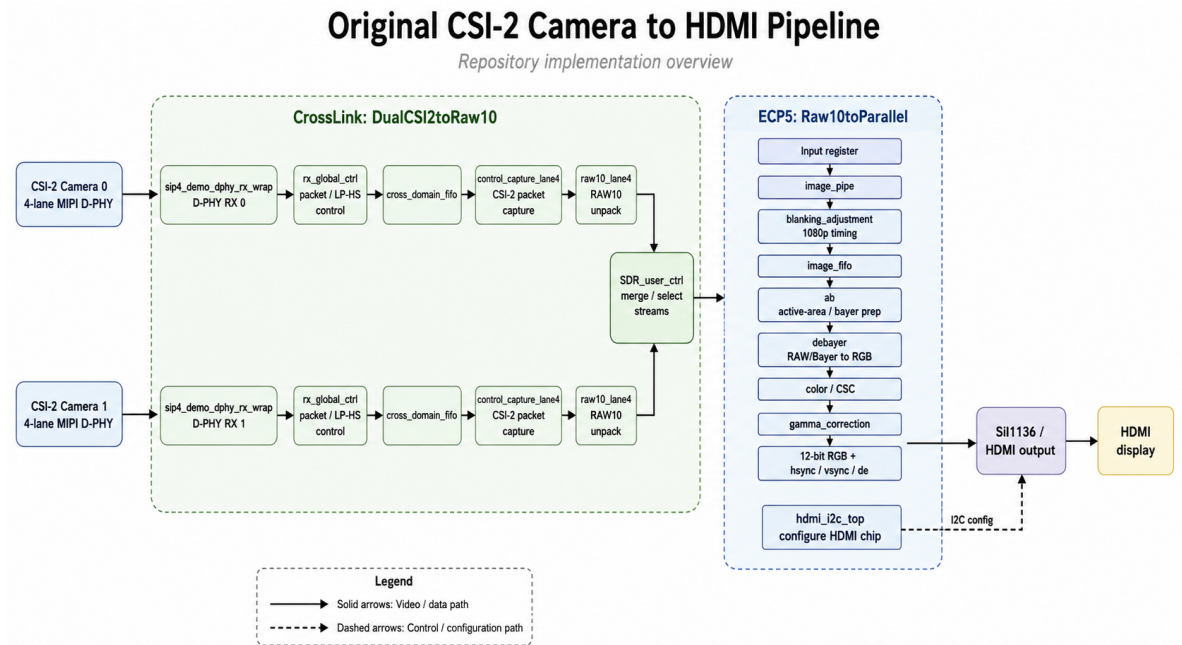


Figure 4.1: Original reference pipeline before custom modifications.

The reference design is useful because it already solves several low-level tasks that are not the main focus of this project. These include camera initialization, video syn-

chronization, HDMI generation, and board-specific constraints. The project therefore concentrates on the processing stage between input decoding and output display.

## 4.2 Pipeline Insertion Point

A key design decision is where to insert the custom processing blocks. The insertion point should satisfy the following conditions:

- Pixel data must be available in a convenient format.
- Synchronization signals must be accessible and preserved.
- The block should not disturb the HDMI timing.
- The inserted logic should be easy to bypass for debugging.

In the project, the custom image-processing logic is placed after the initial camera/color-processing stages and before the final HDMI output path. In earlier experiments, color-space conversion and gamma-related blocks were modified or bypassed to create a cleaner insertion point for custom processing.

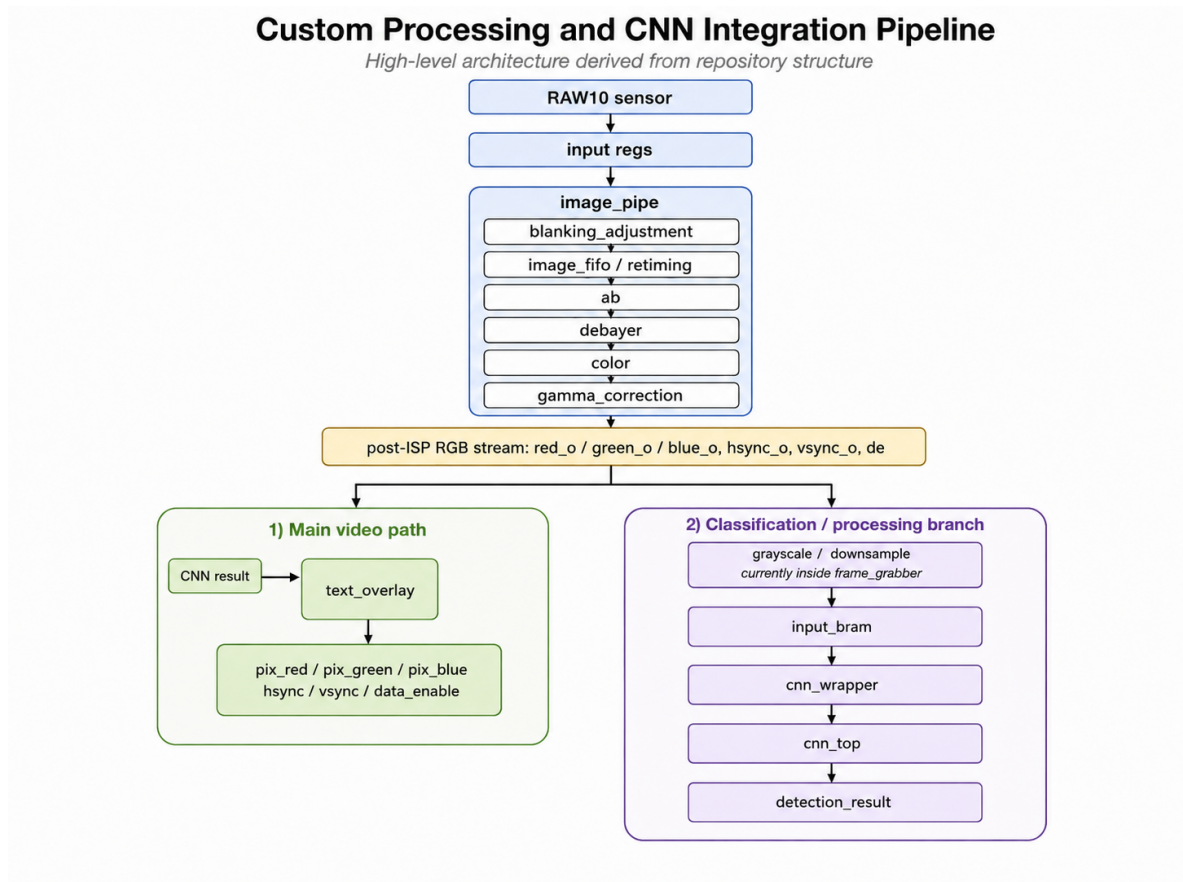


Figure 4.2: Modified pipeline with custom processing insertion point.

## 4.3 Signal Preservation

When modifying a video stream, it is not enough to process RGB or grayscale pixel values. The control signals must also be preserved. A typical video stream contains signals such as:

- pixel clock;
- reset;
- data enable;
- horizontal synchronization;
- vertical synchronization;
- frame start and line start indicators;
- pixel coordinates or counters.

If the pixel data is delayed by a processing block, the control signals must be delayed by the same number of cycles. Otherwise, the display may show shifted images, unstable frames, incorrect colors, or no output. This is especially relevant for filters such as Sobel edge detection, where line buffers and window logic introduce latency.

## 4.4 Debugging Strategy

The debugging strategy is based on preserving a bypass mode. A bypass mode makes it possible to switch between the original video stream and the processed stream. This is useful for distinguishing between two classes of problems:

1. Problems caused by the board, camera, HDMI output, or base design.
2. Problems caused by the newly inserted processing block.

If the bypass output works but the processed output fails, the error is likely inside the custom block or its timing alignment. If both fail, the error is probably in the integration, constraints, clocking, or reference pipeline.

# Chapter 5

## Implemented Image-Processing Blocks

### 5.1 Motivation for Simple Processing Blocks

Before integrating the CNN accelerator, simpler image-processing blocks were implemented. This is an important engineering choice. A CNN accelerator is difficult to debug because it includes arithmetic, memory, control, and interface logic. By contrast, grayscale conversion and thresholding are easier to observe directly on a monitor. If these blocks work correctly, they provide confidence that the insertion point and signal alignment are correct.

The implemented blocks are:

- bypass mode;
- grayscale conversion;
- binary thresholding;
- Sobel edge detection;
- optional overlay path for displaying detection results.

### 5.2 Bypass Mode

The bypass mode directly forwards the incoming pixel stream to the output. It is the simplest possible mode but one of the most important for debugging. It verifies that the modified project can still reproduce the original video output.

Listing 5.1: Simplified bypass logic.

```

1  always @(posedge CSI2_sens_clk or negedge reset_n) begin
2      if (!reset_n) begin
3          pix_red <= 12'd0;
4          pix_green <= 12'd0;
5          pix_blue <= 12'd0;
6          hsync <= 1'b0;
7          vsync <= 1'b0;
8          data_enable <= 1'b0;
9      end else begin
10         pix_red <= overlay_red;
11         pix_green <= overlay_green;
12         pix_blue <= overlay_blue;
13         hsync <= overlay_hsync;
14         vsync <= overlay_vsync;
15         data_enable <= overlay_de;
16     end
17 end

```

In the actual project, the control signals must be forwarded together with the pixel data. The simplified code above only illustrates the pixel path.

### 5.3 Grayscale Conversion

Grayscale conversion reduces RGB data to a single intensity channel. This is useful because the CNN model is trained on grayscale images, and it also reduces hardware complexity. A common approximation is a weighted sum:

$$Y = 0.299R + 0.587G + 0.114B. \quad (5.1)$$

In hardware, floating-point coefficients are avoided. The operation can be approximated using integer arithmetic:

$$Y \approx \frac{77R + 150G + 29B}{256}. \quad (5.2)$$

This expression is efficient because division by 256 corresponds to an 8-bit right shift. The result is not exactly equal to the floating-point expression, but it is accurate enough for many video-processing tasks.

Listing 5.2: Integer approximation for grayscale conversion.

```

1 wire [15:0] gray_sum;
2 assign gray_sum = 8'd77 * in_r +
3                 8'd150 * in_g +
4                 8'd29 * in_b;
5
6 assign gray = gray_sum[15:8]; // divide by 256

```

## 5.4 Thresholding

Thresholding converts a grayscale image into a binary image. If the pixel intensity is above a threshold, the output becomes white; otherwise, it becomes black:

$$B(x, y) = \begin{cases} 255, & Y(x, y) > T, \\ 0, & \text{otherwise.} \end{cases} \quad (5.3)$$

This block is useful for testing because it creates a clear visual output. It also demonstrates how parameterized processing can be controlled through constants or switches.

Listing 5.3: Simplified thresholding logic.

```

1 parameter THRESHOLD = 8'd100;
2 assign binary_pixel = (gray > THRESHOLD) ? 8'hFF : 8'h00;

```

## 5.5 Sobel Edge Detection

Sobel edge detection estimates the image gradient using two convolution kernels:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}. \quad (5.4)$$

The edge magnitude can be approximated as:

$$G \approx |G_x| + |G_y|. \quad (5.5)$$

A Sobel filter requires a  $3 \times 3$  pixel window. In a streaming FPGA implementation, this window is generated using line buffers and shift registers. This makes Sobel more complex than grayscale conversion, but still much simpler than a CNN accelerator.

## 5.6 Lessons from Image-Processing Integration

The simple image-processing blocks provide several important lessons:

- Pixel data and synchronization signals must be delayed consistently.
- A bypass path is essential for debugging.
- Even simple filters can introduce latency that must be handled explicitly.
- Visual output through HDMI is a practical validation method.
- Stream-based processing differs significantly from frame-based software processing.

These lessons directly inform the later CNN integration, where latency, buffering, and synchronization become more difficult.



# Chapter 6

## Tiny CNN for Person Detection

### 6.1 Problem Formulation

The machine-learning part of the project is formulated as a binary classification task. The input is a small grayscale image patch, and the output is a probability-like score indicating whether the patch contains a person. The two classes are:

- **person**: the image patch contains a person or a significant part of a person;
- **non-person**: the image patch does not contain a person.

This formulation is simpler than full object detection. A complete detector would need to estimate bounding boxes and handle multiple people at different scales. The goal here is more limited: to evaluate the feasibility of a tiny CNN inference block on the FPGA.

### 6.2 Dataset Preparation

The dataset was balanced between person and non-person samples. The final dataset contained approximately 903 person images and 903 non-person images, for a total of 1806 samples. The images were converted to grayscale and resized to a small input resolution suitable for FPGA inference.

Table 6.1: Dataset summary.

| Class      | Number of Samples |
|------------|-------------------|
| Person     | 903               |
| Non-person | 903               |
| Total      | 1806              |

## 6.3 Model Architecture

The CNN is intentionally compact. The design objective is not maximum classification accuracy, but a reasonable trade-off between accuracy and hardware complexity. The model uses grayscale input, average pooling, small convolution layers, and a final dense output layer.

A representative architecture is shown in Table 6.2.

Table 6.2: Tiny CNN architecture used for FPGA-oriented person classification.

| Layer           | Output Shape            | Parameters                                       | Purpose                 |
|-----------------|-------------------------|--------------------------------------------------|-------------------------|
| Input           | $32 \times 32 \times 1$ | –                                                | Grayscale image patch   |
| Average Pooling | $16 \times 16 \times 1$ | $2 \times 2$ pooling                             | Reduce input resolution |
| Conv2D + ReLU   | $8 \times 8 \times 1$   | 1 filter, $3 \times 3$ , stride 2, same padding  | Extract simple local f  |
| Conv2D + ReLU   | $4 \times 4 \times 2$   | 2 filters, $3 \times 3$ , stride 2, same padding | Extract higher-level s  |
| Average Pooling | $1 \times 1 \times 2$   | $4 \times 4$ pooling                             | Collapse spatial dimer  |
| Flatten         | 2                       | –                                                | Convert feature maps    |
| Dense + Sigmoid | 1                       | binary output                                    | Person probability      |

The number of filters is deliberately very small. A conventional CNN would normally use many more channels, but that would increase the number of multipliers, adders, memories, and routes required on the FPGA.

## 6.4 Keras Model Definition

A simplified Keras-style model definition is shown below.

Listing 6.1: Representative Keras model structure.

```

1 from tensorflow.keras import layers, models
2
3
4 model = tf.keras.Sequential([
5     tf.keras.layers.Input(shape=(32, 32, 1)),
6     tf.keras.layers.AveragePooling2D(pool_size=(2, 2)),
7
8     tf.keras.layers.Conv2D(
9         filters=1,
10        kernel_size=3,
11        strides=2,
```

```

12         padding="same",
13         activation="relu",
14     ),
15     tf.keras.layers.Conv2D(
16         filters=2,
17         kernel_size=3,
18         strides=2,
19         padding="same",
20         activation="relu",
21     ), tf.keras.layers.AveragePooling2D(pool_size=(4, 4)),
22     tf.keras.layers.Flatten(),
23     tf.keras.layers.Dense(1, activation="sigmoid"),
24 ])

```

## 6.5 Training Results

The trained model achieved good classification performance for its size. One of the final evaluations produced approximately the confusion matrix shown in Table 6.3.

Table 6.3: Confusion matrix obtained on the evaluation set using a classification threshold of 0.50.

|                   | Predicted Non-person | Predicted Person |
|-------------------|----------------------|------------------|
| Actual Non-person | 165                  | 11               |
| Actual Person     | 35                   | 186              |

The corresponding accuracy was approximately 93.7%. The person class had high recall, while the non-person class had slightly lower recall because some background patches were classified as person. For an embedded demonstrator, this behavior is acceptable because the goal is feasibility and real-time integration rather than production-grade detection accuracy.

“`latex`

Table 6.4: Representative classification metrics.

| Class      | Precision | Recall | F1-score | Support |
|------------|-----------|--------|----------|---------|
| Non-person | 0.98      | 0.98   | 0.98     | 176     |
| Person     | 0.98      | 0.98   | 0.98     | 221     |

““

## 6.6 Why the Model Must Be Small

The model is small because the FPGA already contains the camera-to-HDMI pipeline. The CNN is not implemented on an empty device. It must coexist with the reference design and with additional video-processing blocks. A larger model could improve classification accuracy, but would also increase:

- the number of multiplications;
- intermediate activation memory;
- control complexity;
- routing density;
- timing pressure;
- compilation time.

The final routing problem confirms that even a small network can be challenging when integrated into a complete video design.

# Chapter 7

## Fixed-Point Conversion and Hardware Generation

### 7.1 Need for Quantization

The Keras model is trained in floating point, but the FPGA implementation should use fixed-point arithmetic. Floating-point operators are possible on FPGAs, but they are expensive and unnecessary for a tiny binary classifier. Fixed-point arithmetic reduces area and can improve timing when chosen carefully.

The conversion process has two main objectives:

1. Preserve the classification behavior of the floating-point model.
2. Reduce the hardware cost of arithmetic and storage.

### 7.2 Precision Selection

Different fixed-point precisions were evaluated during the conversion process. The final configuration used different bit widths for different layers. A representative precision configuration is shown in Table [7.1](#).

Table 7.1: Representative fixed-point precision configuration.

| Layer / Signal         | Precision   | Reason                                               |
|------------------------|-------------|------------------------------------------------------|
| Input                  | fixed<12,4> | Enough range for normalized grayscale input          |
| First average pooling  | fixed<12,4> | Low-cost downsampling after the input                |
| First convolution      | fixed<16,5> | Preserve feature precision during early convolution  |
| First ReLU output      | fixed<14,4> | Reduce activation bit-width after nonlinearity       |
| Second convolution     | fixed<16,5> | Keep enough range for accumulated convolution values |
| Second ReLU output     | fixed<12,4> | Compact activation representation for hardware       |
| Second average pooling | fixed<20,6> | Provide extra range for pooling accumulation         |
| Flatten                | fixed<12,4> | Vector reshaping without increasing precision cost   |
| Dense layer            | fixed<14,4> | Preserve final classification precision              |
| Sigmoid output         | fixed<12,4> | Binary decision score between person and non-person  |

### 7.3 Floating-Point vs Fixed-Point Agreement

After quantization, the fixed-point model must be compared with the floating-point Keras model. This comparison is more informative than accuracy alone because it reveals whether the conversion changed the decision boundary. A high agreement means that most samples receive the same classification before and after conversion.

A representative comparison showed very high agreement between the two models. The mean absolute error between outputs was small, and only a small number of samples changed class. This indicates that fixed-point conversion was not the main cause of the final hardware difficulty. Instead, the difficulty appears later in physical implementation.

Table 7.2: Representative floating-point vs fixed-point comparison.

| Metric                  | Value  |
|-------------------------|--------|
| Agreement               | 0.9972 |
| Mean absolute error     | 0.0053 |
| Number of disagreements | 5      |

### 7.4 hls4ml Conversion Flow

The hls4ml flow converts the trained neural network into HLS-oriented code. The main steps are:

1. Load the trained Keras model.
2. Create an hls4ml configuration.
3. Specify fixed-point precision for layers and activations.
4. Generate the HLS project.
5. Validate the HLS output against the Keras model.
6. Export C/C++ code for downstream synthesis.

Listing 7.1: Representative hls4ml conversion flow used for the Bambu FPGA implementation.

```
1 import hls4ml
2
3 PART = "LFE5UM85F8BG756C"
4 BACKEND = "bambu"
5 PROJECT_NAME = "myproject"
6
7 # Selected fixed-point precision for each layer/signal
8 SELECTED_LAYER_PRECISIONS = {
9     "input_1": "fixed<12,4>",
10    "average_pooling2d": "fixed<12,4>",
11
12    "conv2d": "fixed<16,5>",
13    "conv2d_relu": "fixed<14,4>",
14
15    "conv2d_1": "fixed<16,5>",
16    "conv2d_1_relu": "fixed<12,4>",
17
18    "average_pooling2d_1": "fixed<20,6>",
19
20    "flatten": "fixed<12,4>",
21    "dense": "fixed<14,4>",
22    "dense_sigmoid": "fixed<12,4>",
23 }
24
25 # Create hls4ml configuration from the trained Keras model
26 config = hls4ml.utils.config_from_keras_model(
27     model,
28     granularity="name",
29     default_precision="fixed<20,6>",
```

```
30     backend=BACKEND,
31 )
32
33 # Global model-level configuration
34 config["Model"]["Precision"] = "fixed<20,6>"
35 config["Model"]["ReuseFactor"] = 9
36 config["Model"]["Strategy"] = "Resource"
37 config["Model"]["IOType"] = "io_parallel"
38
39 # Apply selected per-layer precisions
40 for layer_name, precision in SELECTED_LAYER_PRECISIONS.items():
41     if layer_name in config["LayerName"]:
42         layer_cfg = config["LayerName"][layer_name]
43
44         if isinstance(layer_cfg["Precision"], dict):
45             for key in layer_cfg["Precision"]:
46                 layer_cfg["Precision"][key] = precision
47         else:
48             layer_cfg["Precision"] = precision
49
50 # Convert the Keras model to an hls4ml project
51 hls_model = hls4ml.converters.convert_from_keras_model(
52     model,
53     hls_config=config,
54     output_dir="hls_full_model_simple32",
55     project_name=PROJECT_NAME,
56     backend=BACKEND,
57     part=PART,
58     io_type="io_parallel",
59 )
60
61 # Generate the HLS project and firmware files
62 hls_model.write()
63 hls_model.build(
64     reset=False,
65     csim=False,
66     synth=False,
67 )
68
69 # After project generation, the script patches the generated Bambu files:
70 # - replace unsupported valid interfaces with fifo interfaces
```



```
71  
72 # - verify that the generated C++ input shape is 32x32x1
```

## 7.5 Bambu HLS Flow

Bambu is used to synthesize the generated C/C++ description into RTL. The HLS process creates datapath and control logic. The datapath contains arithmetic units and storage elements, while the control unit schedules operations and manages the sequence of computation.

A representative Bambu command structure is shown below.

Listing 7.2: Representative Bambu synthesis command.

```
1 bambu firmware.cpp \  
2   --top-fname=myproject \  
3   --device-name=LFE5UM-85F \  
4   --clock-period=10 \  
5   --generate-tb \  
6   --simulate \  
7   -o bambu_output
```

## 7.6 Generated Interface

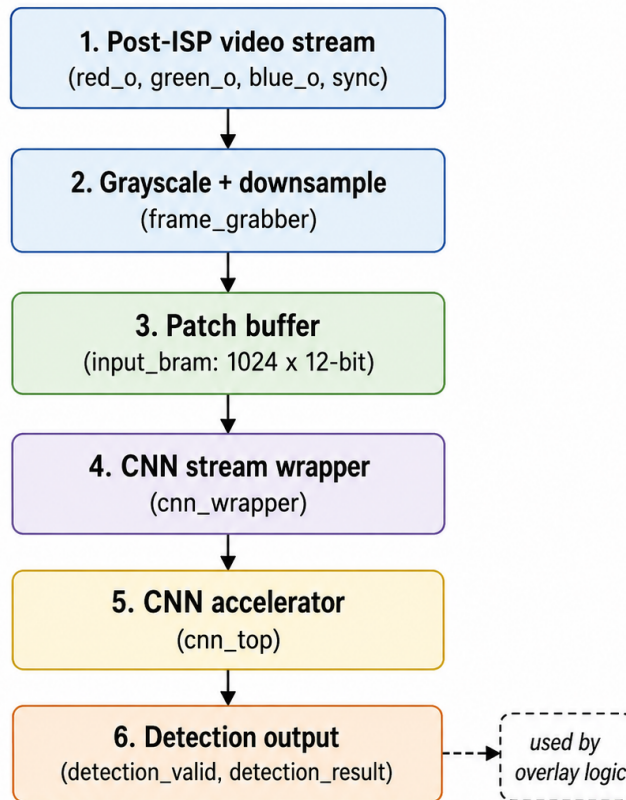
The generated CNN hardware must communicate with the video pipeline. In principle, the interface can be organized in different ways:

- frame-based input, where a full patch is collected before inference;
- streaming input, where pixels are pushed into the accelerator sequentially;
- memory-mapped input, where the accelerator reads from a buffer;
- FIFO-based input, where the pipeline and accelerator communicate through queues.

For this project, a FIFO-style or patch-buffered interface is the most practical conceptually. The video pipeline produces pixels continuously, while the CNN expects a small fixed-size input. Therefore, an adapter is needed between the stream and the CNN module.

## CNN Insertion Point in the Video Pipeline

*Post-ISP stream to detection output*



**Figure 5.x:** CNN insertion point in the video pipeline. The post-ISP RGB stream is converted to a downsampled grayscale patch, buffered in BRAM, streamed into the CNN accelerator, and converted into a detection result used by the overlay logic.

Figure 7.1: Conceptual interface between the streaming video pipeline and CNN inference. The post-ISP RGB stream is converted into a downsampled grayscale patch, buffered in BRAM, streamed to the CNN accelerator, and converted into a detection result used by the overlay logic.

# Chapter 8

## FPGA Integration

### 8.1 Overall Integrated Architecture

The intended final architecture combines the original video pipeline, preprocessing logic, CNN inference, and an output visualization mechanism. The high-level flow is:

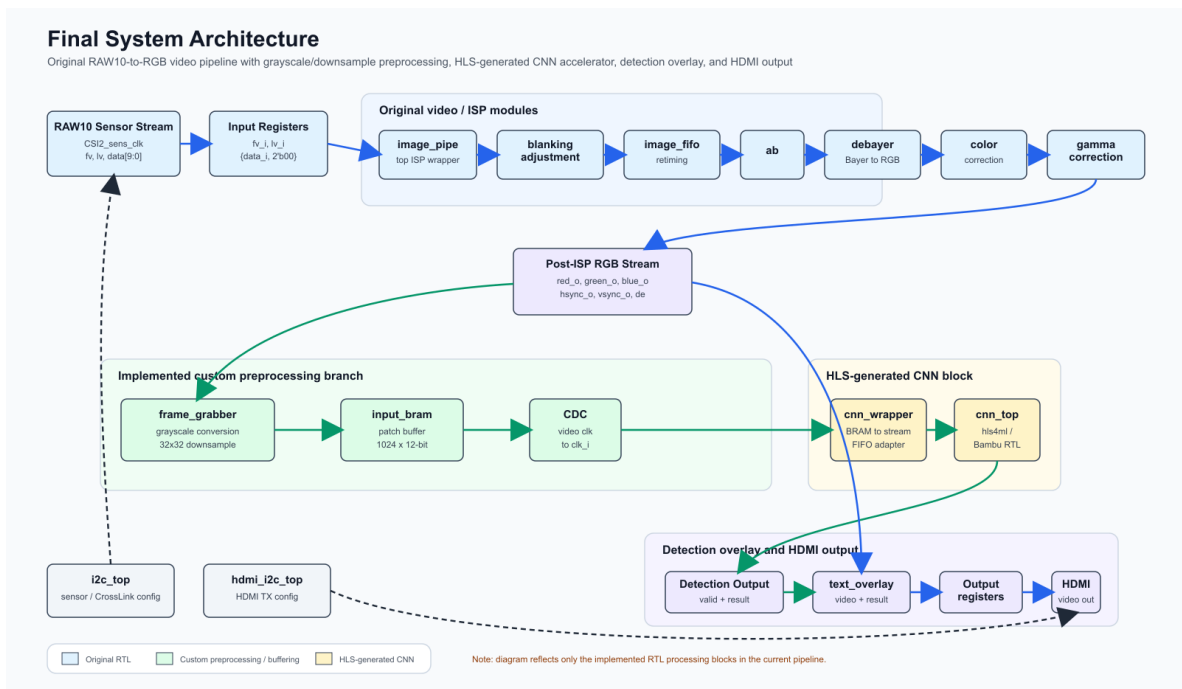


Figure 8.1: Target final architecture of the embedded vision system, including the original video-processing pipeline, custom preprocessing blocks, the HLS-generated CNN accelerator, and the HDMI output stage.

The CNN output can be used in multiple ways. In the simplest case, it controls an LED or a small overlay on the video output. A more advanced version would draw a bounding box or heatmap. Because the CNN in this project is a classifier rather than a full detector, the overlay should be interpreted as a detection indicator rather than a

precise object-localization result.

## 8.2 Patch Extraction

The CNN expects a compact  $32 \times 32 \times 1$  grayscale input, while the live video stream is  $1920 \times 1080$  RGB after the ISP pipeline. Therefore, the FPGA design must reduce the full-resolution video stream to a small CNN-compatible input.

In this implementation, the post-ISP RGB stream is sampled by the `frame_grabber` module. This module converts RGB pixels to grayscale and performs spatial downsampling to generate a  $32 \times 32$  image. The result is a set of 1024 grayscale samples, which are written into the input BRAM and later streamed into the CNN accelerator.

Other strategies are possible, such as:

1. extracting a fixed region of interest from the frame,
2. scanning multiple windows across the frame,
3. using a lightweight pre-filter to select candidate regions.

For this first FPGA demonstrator, full-frame downsampling is the most practical approach because it requires only one CNN evaluation per frame. In contrast, sliding-window detection would require many CNN evaluations per frame and would significantly increase computation, memory bandwidth, and control complexity.

Therefore, the implemented design should not be described as extracting a generic small patch from the frame. More precisely, it generates a  $32 \times 32$  grayscale downsampled representation of the video frame using the `frame_grabber` module.

## 8.3 Latency Alignment

Each processing block introduces latency. The normal video path is implemented as a streaming pipeline, so RGB data, horizontal synchronization, vertical synchronization, and data-enable signals must remain aligned as they pass through the ISP and overlay stages. In this design, the overlay logic delays the video control signals together with the pixel data so that the displayed text remains spatially aligned with the HDMI stream.

The CNN path has a much larger effective latency than the normal video path. The `frame_grabber` module collects a  $32 \times 32$  grayscale downsampled representation of a video frame and writes the 1024 samples into BRAM. After the image patch has been captured, the `cnn_wrapper` streams the data into the HLS-generated CNN accelerator.

As a result, the detection output becomes available only after the corresponding frame has already continued through the live video pipeline.

For this demonstrator, the CNN result is treated as a frame-level status signal. The latest valid detection result is stored and used by the `text_overlay` module on subsequent live video frames. Therefore, the overlay does not modify the exact same frame from which the CNN input was extracted. Instead, it displays the most recent classification result. This approach is acceptable for a real-time demonstration where the output is a global detection label rather than a pixel-accurate bounding box.

A more advanced design requiring exact frame or region correspondence would need additional synchronization mechanisms, such as frame identifiers, delayed video buffering, region-of-interest metadata, or full-frame storage. These mechanisms would increase memory usage and control complexity, so they were avoided in the first FPGA demonstrator.

## 8.4 Clocking and Reset

The design uses multiple clock domains. The live video pipeline runs in the `CSI2_sens_clk` domain, while the CNN accelerator and its wrapper run in the `clk_i` domain. Therefore, signals that pass between the video path and the CNN path require explicit clock-domain crossing logic.

The input patch buffer is implemented as a dual-clock BRAM. The `frame_grabber` writes the  $32 \times 32$  grayscale patch in the video clock domain, and the `cnn_wrapper` reads the stored samples in the `clk_i` domain. Control signals are synchronized separately. The `frame_done` pulse is converted into a toggle and synchronized into the CNN clock domain, where it starts the CNN readout process. Similarly, the CNN detection result is latched in the `clk_i` domain and transferred back to the video clock domain using a toggle-based synchronizer.

Reset behavior is also important. Most RTL modules use the active-low `reset_n` signal. The generated CNN block exposes a port named `reset`; however, in this generated design the reset is also active-low, so it is connected directly to `reset_n`. The CNN accelerator is controlled through a start/done interface and FIFO-style handshaking signals, including `input_1_empty_n`, `input_1_read`, `layer10_out_full_n`, and `layer10_out_write`. These signals must be connected correctly so that the accelerator receives exactly one complete patch and the output result is captured only when valid. Incorrect clock-domain crossing or reset behavior can cause missed frame-start events, stale CNN outputs, repeated classifications, or an accelerator that never starts. For this reason, the design separates data transfer through BRAM from control transfer

through explicit synchronizers.

## 8.5 Generated RTL Integration

The RTL generated by Bambu/hls4ml must be added to the Lattice Diamond project together with any support library files required by the generated design. In this project, the generated CNN logic is included through the Bambu-generated `myproject.v` file, the supporting `panda_libtech.v` file, and a small local wrapper called `cnn_top.v`.

The integration requires:

- adding the generated Verilog and support files to the Diamond project;
- wrapping the long generated module name behind a simple local module;
- connecting the CNN clock and active-low reset;
- adapting the input data width to the 12-bit fixed-point CNN input;
- connecting the FIFO-style input handshake signals;
- connecting the output write signal and 12-bit result;
- checking synthesis warnings and unresolved module references;
- verifying that the generated hierarchy is preserved correctly.

The generated RTL contains many internal modules and long tool-generated names. To keep the rest of the system readable, the generated CNN is isolated behind `cnn_top`. A separate `cnn_wrapper` module then handles the system-level control, BRAM addressing, input streaming, and output result capture.

Listing 8.1: Local wrapper around the Bambu-generated CNN module.

```
1 module cnn_top (  
2     input wire clock,  
3     input wire reset, // Active-low in generated RTL  
4     input wire start_port,  
5  
6     input wire [11:0] input_1_dout,  
7     input wire input_1_empty_n,  
8     output wire input_1_read,  
9  
10    input wire layer10_out_full_n,  
11    output wire [11:0] layer10_out_din,
```

```

12     output wire layer10_out_write,
13
14     output wire done_port
15 );
16
17     // Instantiates the Bambu-generated CNN module with its
18     // long tool-generated module name.
19
20 endmodule

```

The `cnn_top` module only hides the tool-generated CNN interface behind a cleaner local module name. The higher-level `cnn_wrapper` sits outside this block and converts the stored BRAM patch into the streaming interface expected by the CNN accelerator. It reads the 1024 grayscale samples from `input_bram`, drives the input handshake signals, starts the accelerator, waits for `done_port`, and captures the 12-bit output result when `layer10_out_write` is asserted.

This separation makes the integration easier to debug. The Bambu-generated block remains isolated, while the rest of the video system only interacts with a controlled wrapper interface.

## 8.6 Overlay Output

The visual output is implemented as a simple text overlay on the HDMI video stream. Instead of drawing a precise bounding box, the design displays either `DETECTED` or `NOT DETECTED` in the top-left corner of the image. This is simpler than object-localized detection, but it still demonstrates that the CNN inference result affects the live video output.

The CNN output is compared against a fixed-point threshold:

$$\text{detected} = \begin{cases} 1, & \text{score} > \theta, \\ 0, & \text{otherwise.} \end{cases} \quad (8.1)$$

where  $\theta$  is the detection threshold. In this implementation, the threshold is set to `12'h080`, corresponding to 0.5 in the chosen fixed-point output format. Scores below this threshold are interpreted as the detected class.

The overlay logic writes the detection status as text on top of the video stream. The background of the text region is black. When a person is detected, the displayed `DETECTED` text is green. When no person is detected, the displayed `NOT DETECTED` text is white.

The overlay logic also includes a small hysteresis mechanism. The displayed state changes only after the same new detection state has been observed for multiple CNN results. This reduces flickering when the CNN score is close to the threshold. The video data and synchronization signals are delayed through the overlay pipeline so that the text pixels remain aligned with the HDMI output stream.